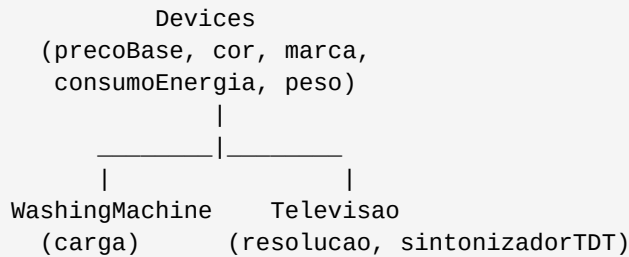


Exercício POO — Herança em Java

Diagrama de Herança



1. Classe Devices (Superclasse)

```
public class Devices {
    // Constantes padrão
    protected static final String COR_PADRAO = "branco";
    protected static final char CONSUMO_PADRAO = 'F';
    protected static final double PRECO_PADRAO = 100.0;
    protected static final double PESO_PADRAO = 5.0;

    // Atributos
    protected double precoBase;
    protected String cor;
    protected String marca;
    protected char consumoEnergia;
    protected double peso;

    // Construtor geral
    public Devices() {
        this.precoBase = PRECO_PADRAO;
        this.cor = COR_PADRAO;
        this.marca = "";
        this.consumoEnergia = CONSUMO_PADRAO;
        this.peso = PESO_PADRAO;
    }

    // Construtor com preco e peso
    public Devices(double preco, double peso) {
        this.precoBase = preco;
        this.peso = peso;
        this.cor = COR_PADRAO;
        this.marca = "";
        this.consumoEnergia = CONSUMO_PADRAO;
    }

    // Construtor com todos os atributos
    public Devices(double precoBase, String cor, String marca,
        char consumoEnergia, double peso) {
        this.precoBase = precoBase;
        this.cor = verificarColor(cor);
        this.marca = marca;
        this.consumoEnergia = checkEnergyConsumption(consumoEnergia);
        this.peso = peso;
    }
}
```

```

// Gets
public double getPrecoBase()      { return precoBase; }
public String getCor()            { return cor; }
public String getMarca()          { return marca; }
public char getConsumoEnergia()   { return consumoEnergia; }
public double getPeso()           { return peso; }

// Verifica consumo de energia (privado)
private char checkEnergyConsumption(char letter) {
    char l = Character.toUpperCase(letter);
    if (l >= 'A' && l <= 'F') return l;
    return CONSUMO_PADRAO;
}

// Verifica cor (privado)
private String verificarColor(String color) {
    String c = color.toLowerCase();
    if (c.equals("branco") || c.equals("preto") || c.equals("vermelho")
        || c.equals("azul") || c.equals("cinza")) {
        return c;
    }
    return COR_PADRAO;
}

// Preco final baseado no consumo de energia
public double precoFinal() {
    double preco = precoBase;
    switch (consumoEnergia) {
        case 'A': preco += 100; break;
        case 'B': preco += 80; break;
        case 'C': preco += 60; break;
        case 'D': preco += 50; break;
        case 'E': preco += 30; break;
        case 'F': preco += 10; break;
    }
    return preco;
}
}

```

2. Classe WashingMachine (Subclasse de Devices)

```

public class WashingMachine extends Devices {
    private static final double CARGA_PADRAO = 5.0;
    private double carga;

    // Construtor geral
    public WashingMachine() {
        super();
        this.carga = CARGA_PADRAO;
    }

    // Construtor com preco e peso
    public WashingMachine(double preco, double peso) {
        super(preco, peso);
        this.carga = CARGA_PADRAO;
    }
}

```

```

// Construtor com todos os atributos
public WashingMachine(double carga, double precoBase, String cor,
                      String marca, char consumoEnergia, double peso) {
    super(precoBase, cor, marca, consumoEnergia, peso);
    this.carga = carga;
}

// Get
public double getCarga() { return carga; }

@Override
public double precoFinal() {
    double preco = super.precoFinal(); // chama metodo da classe mae

    // Preco por tamanho da carga
    if (carga >= 0 && carga <= 19)        preco += 10;
    else if (carga >= 20 && carga <= 49)    preco += 50;
    else if (carga >= 50 && carga <= 79)    preco += 80;
    else if (carga > 80)                    preco += 100;

    // Carga superior a 30 kg
    if (carga > 30) preco += 50;

    return preco;
}
}

```

3. Classe Televisao (Subclasse de Devices)

```

public class Televisao extends Devices {
    private static final int RESOLUCAO_PADRAO = 20;
    private static final boolean SINTONIZADOR_PADRAO = false;

    private int resolucao;
    private boolean sintonizadorTDT;

    // Construtor geral
    public Televisao() {
        super();
        this.resolucao = RESOLUCAO_PADRAO;
        this.sintonizadorTDT = SINTONIZADOR_PADRAO;
    }

    // Construtor com preco e peso
    public Televisao(double preco, double peso) {
        super(preco, peso);
        this.resolucao = RESOLUCAO_PADRAO;
        this.sintonizadorTDT = SINTONIZADOR_PADRAO;
    }

    // Construtor com todos os atributos
    public Televisao(int resolucao, boolean sintonizadorTDT, double precoBase,
                    String cor, String marca, char consumoEnergia, double peso) {
        super(precoBase, cor, marca, consumoEnergia, peso);
        this.resolucao = resolucao;
        this.sintonizadorTDT = sintonizadorTDT;
    }
}

```

```

// Gets
public int getResolucao()          { return resolucao; }
public boolean getSintonizadorTDT(){ return sintonizadorTDT; }

@Override
public double precoFinal() {
    double preco = super.precoFinal(); // chama metodo da classe mae

    // Resolucao superior a 40 polegadas -> +30%
    if (resolucao > 40) preco *= 1.30;

    // Sintonizador TDT -> +50 euros
    if (sintonizadorTDT) preco += 50;

    return preco;
}
}

```

4. Classe Main (Executável)

```

public class Main {
    public static void main(String[] args) {

        // Objeto Devices
        Devices d = new Devices(150.0, "azul", "Samsung", 'B', 8.0);
        System.out.println("=== Device ===");
        System.out.println("Marca: " + d.getMarca());
        System.out.println("Preco final: " + d.precoFinal() + " euros");

        // Objeto WashingMachine
        WashingMachine wm = new WashingMachine(
            35.0, 200.0, "branco", "Bosch", 'A', 70.0);
        System.out.println("\n=== Maquina de Lavar ===");
        System.out.println("Marca: " + wm.getMarca());
        System.out.println("Carga: " + wm.getCarga() + " kg");
        System.out.println("Preco final: " + wm.precoFinal() + " euros");

        // Objeto Televisao
        Televisao tv = new Televisao(
            55, true, 500.0, "preto", "LG", 'C', 15.0);
        System.out.println("\n=== Televisao ===");
        System.out.println("Marca: " + tv.getMarca());
        System.out.println("Resolucao: " + tv.getResolucao() + " polegadas");
        System.out.println("Sintonizador TDT: " + tv.getSintonizadorTDT());
        System.out.println("Preco final: " + tv.precoFinal() + " euros");
    }
}

```

Tabelas de Referência

Preço por Consumo Energético (Devices):

Letra	Preço Adicional
A	100 €
B	80 €
C	60 €

D	50 €
E	30 €
F	10 €

Preço por Carga (WashingMachine):

Tamanho	Preço Adicional
0 a 19 kg	10 €
20 a 49 kg	50 €
50 a 79 kg	80 €
Maior que 80 kg	100 €